P2_L1

Enunciado

Usted está trabajando en una empresa proveedora de servicios y recibe la solicitud de un cliente que desea tener un servidor para la implantación de un comercio electrónico. Una vez diseñado y configurado el almacenamiento, debe enviarle las credenciales de acceso para que el equipo de desarrollo pueda desplegar la aplicación. Dado que FTP no es un protocolo seguro, decide configurar SSH para que el usuario sea capaz de acceder a la consola de manera segura y asegurada (secure and hardened). Para ello, configurará el servicio de modo que el usuario root no pueda tener acceso, que sea posible la autenticación por llave pública, restringirá el usuario que puede acceder acceso y limitará los ataques por fuerza bruta. Además cambiará el puerto por defecto para que no esté en la lista de barrido rápido de nmap

---

Para esta práctica lo primero que debemos hacer es crear una red con las VM creadas en las prácticas anteriores. Las conectamos entre ellas añadiendo un adaptador de red de tipo NAT para que estén conectadas a internet y otro adaptador de red de tipo host-only para que estén conectadas entre ellas. Mediante el comando "ip addr" obtenemos las ips de cada una de las vm y hacemos un ping entre ellas para comprobar que están conectadas correctamente.

Una vez hecho todo esto, empezamos la práctica instalando en Debian ssh mediante el comando "apt install openssh-server".

Openssh-server es un servidor ssh(security shell) que permite conectarse de forma remota y segura a otra máquina Linux desde la terminal. Convierte mi VM en un host al que puedo entrar desde otra VM o desde mi host usando cliente SSH.

Con el comando "systemctl status sshd" comprobamos que sshd está funcionando correctamente.

```
[marcosga@localhost ~]$ systemctl status sshd
● sshd.service - OpenSSH server daemon
     Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: ena>
     Active: active (running) since Sat 2025-10-11 09:33:59 CEST; 52min ago
```

A continuación, utilizamos el comando "systemctl start sshd" para arrancar el servicio. Y después, ejecutamos "systemctl enable sshd" para que el servicio se arranque automáticamente al iniciar el sistema.

Desde la VM AlmaLinux ejecutamos "ssh marcosga@192.168.56.101" siendo "marcosga" mi nombre de usuario y el resto la ip de mi VM Debian. Y utilizamos

el mismo comando en la terminal de Debian pero poniendo la ip de AlmaLinux. Con esto, ya tendremos nuestras VMs conectadas entre si, actuando Debian como servidor ssh y AlmaLinux como cliente ssh.

En la siguiente imagen vemos el resultado de la ejecución del comando desde AlmaLinux:



Una vez instalados nmap e hydra mediante apt. Ejecutamos el comando "nmap (ip de VM AlmaLinux)" y vemos que el puerto 22 está abierto utilizando el servicio ssh.

```
root@debian:/home/marcosga# nmap 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-11 16:31 CEST
Nmap scan report for 192.168.56.102
Host is up (0.00079s latency).
Not shown: 988 filtered tcp ports (no-response), 10 filtered tcp ports (admin-pr
ohibited)
PORT     STATE  SERVICE
22/tcp   open   ssh
9090/tcp closed zeus-admin
MAC Address: 08:00:27:E1:D9:40 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 4.82 seconds
```

Creamos un documento de texto llamado passwords.txt que contenga las posibles contraseñas para acceder a AlmaLinux, entre ellas pongo la contraseña correcta para comprobar que funcione bien.

```
root@debian:/home/marcosga# vi passwords.txt
root@debian:/home/marcosga# cat passwords.txt
45n45,nfd,ui
5jllkgfduio
df98uf98f8
boji4tnlk
fiuvgfjd
practicas,ise
```

Con el comando "hydra -l marcosga -P passwords.txt ssh://(ip de AlmaLinux)" lo que hacemos es enviar una serie de accesos en paralelo al usuario marcosga,

probando una serie de contraseñas puestas previamente en el documento passwords.txt y atacando al servicio ssh://(ip de AlmaLinux) que habíamos observado previamente que estaba abierto mediante el comando nmap.

Accedemos a /etc/ssh/sshd_config mediante vi y donde aparece PermitRootLogin ponemos no, para evitar que root pueda entrar.



Con "ufw status" vemos que el firewall está inactivo porque todavia no lo hemos activado.

Con "ufw enable" encendemos el firewall.

Con "ufw allow 22022" desbloqueamos el firewall para ese puerto específico. En la siguiente captura vemos la ejecución de estos comandos.

```
root@debian:/home/marcosga# ufw status
Status: inactive
root@debian:/home/marcosga# ufw enable
Firewall is active and enabled on system startup
root@debian:/home/marcosga# ufw allow 22022
Rule added
Rule added (v6)
```

Y volvemos a ejecutar "ufw status" para comprobar que hemos dejado pasar el tráfico por ese puerto y no ha habido ningún error.

```
root@debian:/home/marcosga# ufw status
Status: active

To                         Action      From
--                         ------      ----
22022                      ALLOW       Anywhere
22022 (v6)                 ALLOW       Anywhere (v6)
```

Ahora instalamos semanage para administrar los ajustes permanentes de SELinux. Para ellos ejecutamos "dnf provides semanage" que le pide al gestor de paquetes que nos diga qué paquete o archivo contiene el comando semanage.

```
policycoreutils-python-utils-3.8-1.el10.noarch : SELinux policy core python utilities
Repo        : appstream
Matched from:
Filename    : /usr/sbin/semanage
```

Como vemos, nos dice que semanage se encuentra en el paquete policycoreutils-python-utils, así que lo instalamos con "dnf install policycoreutils-python-utils".

Ahora con "semanage port -l | grep ssh" vemos los puertos que están asociados al servicio ssh. Y nos sale que el puerto 22.

Ejecutamos el comando "semanage port -a -t ssh_port_t -p tcp 22022" para agregar el puerto 22022 también al servicio.

```
[marcosga@localhost ~]$ sudo semanage port -l | grep ssh
[sudo] password for marcosga:
ssh_port_t                     tcp      22
[marcosga@localhost ~]$ semanage port -a -t ssh_port_t -p tcp 22022
ValueError: SELinux policy is not managed or store cannot be accessed.
[marcosga@localhost ~]$ sudo semanage port -a -t ssh_port_t -p tcp 22022
[  141.488450] SELinux:  Converting 308 SID table entries...
[  141.494481] SELinux:  policy capability network_peer_controls=1
[  141.494495] SELinux:  policy capability open_perms=1
[  141.494502] SELinux:  policy capability extended_socket_class=1
[  141.494509] SELinux:  policy capability always_check_network=0
[  141.494515] SELinux:  policy capability cgroup_seclabel=1
[  141.494594] SELinux:  policy capability nnp_nosuid_transition=1
[  141.494604] SELinux:  policy capability genfs_seclabel_symlinks=1
[  141.494613] SELinux:  policy capability ioctl_skip_cloexec=0
[  141.494622] SELinux:  policy capability userspace_initial_context=0
```

Comprobamos que este nuevo puerto haya sido añadido.

```
[marcosga@localhost ~]$ sudo semanage port -l | grep ssh
[sudo] password for marcosga:
ssh_port_t                     tcp      22022, 22
```

Abrimos el puerto 22022 para conexiones tcp entrantes en el firewall, esto no afecta a SELinux, solo permite que el tráfico llegue al sistema. Para esto ejecutamos el comando "firewall-cmd --add-port=22022/tcp --permanent" y para comprobarlo ejecutamos el comando "firewall-cmd –list-ports".

```
[marcosga@localhost ~]$ sudo firewall-cmd --list-ports
22022/tcp
```

Hacemos "firewall –reload" para añadir los cambios inmediatamente al firewall que esté activo.

Ahora ejecutamos "systemctl restart ssh" para que los cambios también se añadan a la configuración de ssh, así escucha el nuevo puerto que hemos habilitado.
Desde Debian comprobamos que se haya limitado el acceso mediante el siguiente comando y vemos que pone que está el estado "filtered" que significa que el firewall ha bloqueado el acceso a ese puerto.

```
marcosga@debian:~$ nmap -Pn -p 22022 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-12 13:11 CEST
Nmap scan report for 192.168.56.102
Host is up.

PORT      STATE    SERVICE
22022/tcp filtered unknown

Nmap done: 1 IP address (1 host up) scanned in 2.07 seconds
```

Con hydra vemos que ya no me deja acceder a la contraseña con lo que hemos limitado el acceso.

```
marcosga@debian:~$ hydra -l marcosga -P passwords.txt -s 22022 ssh://192.168.56.
102
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in mi
litary or secret service organizations, or for illegal purposes (this is non-bin
ding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-10-12 13:15:
55
[WARNING] Many SSH configurations limit the number of parallel tasks, it is reco
mmended to reduce the tasks: use -t 4
[DATA] max 6 tasks per 1 server, overall 6 tasks, 6 login tries (l:1/p:6), ~1 tr
y per task
[DATA] attacking ssh://192.168.56.102:22022/
[ERROR] could not connect to ssh://192.168.56.102:22022 - Connection refused
```

En AlmaLinux instalamos fil2ban con el comando “dnf install fail2ban” para vigilar los logs y bloquear a quien intente entrar en repetidamente.

En el archivo “jail.conf” configuramos que port = ssh, logpath = /var/log/auth.log, que el número máximo de intentos permitidos sea 3 con maxretry=3, que el baneo dure una hora “bantime=3600” y que el tiempo para contar los intentos sea de 5 minutos “findtime=300”.

Al intentar acceder varias veces desde Debian ahora se nos banea durante 1 hora.

Si ejecutamos “fail2ban-client status sshd” nos muestra la lista de ip que han sido baneadas junto con más información del baneo.

Para acabar la parte de hardening restringimos el usuario que puede acceder editando /etc/ssh/sshd_config añadiendo AllowUsers marcosga.

```
Port 22022
AllowUsers marcosga
```

Comprobamos con ssh -p 22022 marcosga@localhost que funciona y vemos que si nos deja acceder con el usuario que hemos añadido a AllowUsers.

```
[marcosga@localhost ~]$ ssh -p 22022 marcosga@localhost
marcosga@localhost's password:
Last login: Fri Oct 17 14:36:46 2025
```

Para comprobar que cualquier otro usuario no añadido a AllowUsers es rechazado, he creado otro usuario llamado prueba con el comando “adduser prueba“, le he otorgado algunos permisos y le he añadido una contraseña llamada “practicas,ise“ con el comando “passwd ‘contraseña’ “, y vemos que el permiso es denegado.

```
[marcosga@localhost ~]$ ssh -p 22022 prueba@localhost
prueba@localhost's password:
Permission denied, please try again.
prueba@localhost's password:
Permission denied, please try again.
prueba@localhost's password:
prueba@localhost: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
```

Para añadir la autentificación por clave pública, primero accedemos al archivo sshd_config y añadimos la siguiente línea para evitar que se pueda acceder mediante password.

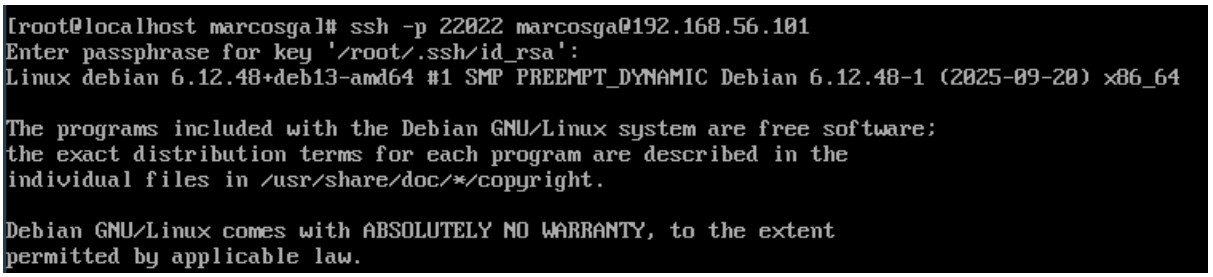
```
PasswordAuthentication no
```

Creamos la clave pública mediante el comando “ssh-keygen -t rsa -b 4096 -C “marcosga@vm”” añadiendo como passphrase “practicas,ise“.

```
[root@localhost marcosga]# ssh-keygen -t rsa -b 4096 -C "marcosga@vm"
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase for "/root/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:gLi15rzCQCYlprGIGhJX/N0niz53v1eVUHCciC48qXs marcosga@vm
The key's randomart image is:
+---[RSA 4096]----+
|   o.       ..=o.|
|+.o...     . o.o |
|=B. o.....o   . .|
|Oo o ....=o..  ..|
|*.. o   S.o+    .|
|o  +   .. .     .|
| o  o  ..       .|
|  o  . .oE. .   .|
|   ..   .o . .oo |
+----[SHA256]-----+
```

Copiamos la llave pública en Debian ejecutando el siguiente comando: “ssh-copy-id -i ~/.ssh/id_rsa.pub -p 22022 marcosga@192.168.56.101“.

Ahora para comprobar que la clave pública funciona correctamente hacemos ssh y vemos que solo nos pide el passphrase, así que funciona todo bien.



Con esto damos por finalizada la P2 L1.